TORQUE & THROTTLE

COMPLETE PROJECT TECHNICAL SPECIFICATION & COMPREHENSIVE STUDY ROADMAP

1. Executive Overview & System Objective

Torque & Throttle is a production-grade luxury automotive vault and dealership management platform. It bridges an immersive client-facing supercar showroom with an enterprise-level dealer portal, hardware-grade dyno telemetry curves, 100-point paint and frame inspections, and an automated omnichannel lead-acquisition pipeline with AI agent webhook relays.

Core Architectural Pillars
Interactive catalog with 0-60, category, transmission & dynamic company filters.

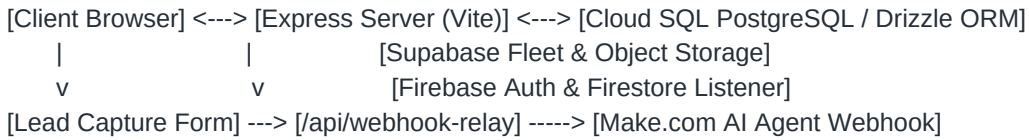
Client Showroom Experience
Dyno horsepower/torque curves, borescope status, paint depth microns inspection.

Dealer Vault Operations
Secured RBAC portal for CRUD inventory updates, canvas photo compression, and status tracking.

Omnichannel Lead Pipeline
FormSubmit email dispatch + server-side Make.com AI Agent webhook relay with auto-retries.

Hybrid Storage Architecture
Dual persistence via PostgreSQL Cloud SQL, Supabase Realtime, and Firebase Firestore.

High-Level System Topology



2. Languages, Frameworks & Runtime Architecture

The platform is engineered using modern web standards prioritizing performance, type safety, and scale-to-zero database infrastructure.

A. Core Programming Languages

- **(*)** TypeScript (~5.8.2): End-to-end type safety across components, REST contracts, and Drizzle models.
- **(*)** JavaScript / Node.js (v22+): Express backend, ES module runtime, async processing.
- **(*)** HTML5 Canvas API: Client-side image resampling and lossy JPEG compression (< 300KB).
- **(*)** Modern CSS3: Utility-first styling via Tailwind CSS v4, dark luxury theme, responsive grids.
- **(*)** SQL (PostgreSQL): Drizzle schema tables, Supabase Realtime, and JSONB telemetry schemas.

B. Frontend Framework Stack

- **(*)** React 19 (19.0.1): Component-driven architecture, custom hooks, useMemo optimization.
- **(*)** Tailwind CSS v4 (4.1.14): High-performance Vite engine, zero-runtime overhead utility CSS.
- **(*)** Lucide React (0.546.0): High-fidelity iconography (gauges, badges, shields, engine symbols).
- **(*)** Motion (12.23.24): Declarative spring animations, modal reveals, and smooth gallery transitions.

C. Backend & Middleware Stack

- **(*)** Express.js (4.21.2): RESTful endpoints for vehicles, enquiries, sync, and webhooks.
- **(*)** Compression (1.8.2): Gzip & Brotli HTTP payload compression minimizing network egress.
- **(*)** Vite 6 Middleware: Runs during development; serves content-hashed static assets in production.
- **(*)** tsx (4.21.0): Zero-configuration TypeScript execution engine for server.ts.

D. Persistence & Storage Engines

- **(*)** PostgreSQL (pg 8.23.0) & Drizzle ORM (0.45.3): Relational data with JSONB for dyno curves.
- **(*)** Supabase (@supabase/supabase-js 2.117.2): Postgres database, bucket storage, realtime events.
- **(*)** Firebase Firestore & Auth (12.19.0): Role-Based Access Control (RBAC) and real-time syncing.
- **(*)** Make.com & FormSubmit: Server-side webhook relays for automated dealer lead automation.

3. Comprehensive Project Directory & Module Breakdown

Every directory and primary file in the repository serves a distinct operational purpose:

A. Core Entry Points & Backend Routes

- **(*)** /server.ts: Express API runner, auth validation, proxy endpoints, and webhook relays.
- **(*)** /src/main.tsx & /src/index.css: React DOM entry point and Tailwind CSS styles.
- **(*)** /src/App.tsx: Master hash router, modal controller, and real-time Firestore listeners.
- **(*)** /src/types.ts: TypeScript interfaces for Vehicle, VehicleSpecs, DynoTelemetry, Enquiry.
- **(*)** /src/data/inventory.ts: Static seed dataset featuring sports cars, SUVs, and luxury sedans.

B. Client-Facing Showroom Components (/src/components)

- **(*)** Navbar.tsx: Sticky navigation bar with live search input and direct quick links.
- **(*)** Hero.tsx: High-impact hero section with dynamic count of available and delivered vault cars.
- **(*)** InventoryPage.tsx & VehicleGrid.tsx: 3-column responsive catalog with dynamic filtering.
- **(*)** FilterBar.tsx: Category tabs (Sedan, SUV, Hatchback, MPV), make filter, and gearbox select.
- **(*)** VehicleDetailPage.tsx: Dedicated full-page view with interactive dyno curves and specs.
- **(*)** VehicleDetailModal.tsx & VehicleImageSlider.tsx: High-res gallery slider with thumbnails.
- **(*)** BookingForm.tsx & EnquiryModal.tsx: Test drive scheduling and vehicle enquiry lead forms.
- **(*)** CustomRequestBanner.tsx & CustomRequestModal.tsx: Bespoke supercar sourcing requests.
- **(*)** HowItWorks.tsx, TrustBadges.tsx, Footer.tsx: Verification guarantees and brand information.

C. Administrative Vault Portal (/src/components/admin)

- **(*)** AdminLogin.tsx: Authenticated entry screen enforcing admin credentials.
- **(*)** AdminDashboard.tsx: Master-detail fleet manager, telemetry index, and batch status actions.
- **(*)** AddVehicleModal.tsx & EditVehicleModal.tsx: 13-parameter car registration forms with photo uploads.
- **(*)** TelemetryInspector.tsx: Inspection auditor verifying brake pads, paint microns, and ECU flashes.
- **(*)** SupabaseSettingsModal.tsx: Live credentials configuration, database sync, and test diagnostics.

D. Database & Services Architecture (/src/db, /src/firebase, /src/supabase)

- **(*)** /src/db/schema.ts: Drizzle tables for vehicles (with JSONB telemetry), enquiries, and users.
- **(*)** /src/db/vehicles.ts & /src/db/enquiries.ts: CRUD queries with fallback to in-memory datasets.
- **(*)** /src/db/seed.ts: Populates database tables with certified vehicle records.
- **(*)** /src/firebase/authService.ts: Admin authentication state listener and session storage handler.
- **(*)** /src/firebase/vehicleService.ts: Facade unifying Supabase, Firestore, and local memory caches.
- **(*)** /src/supabase/supabaseVehicleService.ts: Supabase row mapping, upserts, and real-time streams.
- **(*)** /src/supabase/supabaseStorageService.ts: Cloud image bucket uploading with UUID keys.

E. Utility Engines (/src/utils)

- **(*)** /src/utils/imageCompressor.ts: Browser-side Canvas compression keeping uploads under 300KB.
- **(*)** /src/utils/mailDispatcher.ts: Dual-dispatch lead engine sending FormSubmit & Make.com webhooks.
- **(*)** /src/middleware/auth.ts: Validates Bearer tokens on protected backend API routes.

4. Key System Workflows Explained

Customer Enquiry & Lead Automation Workflow

1. Customer submits BookingForm or EnquiryModal with desired time and contact method.
2. Frontend invokes submitEnquiryLead() in mailDispatcher.ts with normalized payload.
3. Payload is sent to Express backend endpoint /api/enquiries for database persistence.
4. server.ts initiates an outbound HTTP POST to Make.com webhook with a 12s timeout.
5. If attempt 1 encounters network timeout, server automatically executes attempt 2.
6. Customer receives an instant verification reference ID with zero UI interruption.

Photo Upload & Canvas Compression Workflow

1. Dealer selects high-resolution camera photos (e.g. 10MB+) in AddVehicleModal.
2. imageCompressor.ts resizes image to max 1200x800 resolution maintaining aspect ratio.
3. Browser Canvas exports JPEG at 70% quality, reducing file size by over 90%.
4. Compressed Blob is transmitted to Supabase / Firebase storage bucket without latency.

5. Complete Step-by-Step Study Roadmap

Follow this structured 6-phase curriculum to understand and master every architectural layer:

Phase 1: Domain Modeling & TypeScript Contracts (Day 1)
Study src/types.ts and src/data/inventory.ts. Master how Vehicle, DynoTelemetry, and Enquiry are modeled.

Phase 2: Backend Architecture & Webhook Relays (Day 2)
Inspect server.ts, compression middleware, and the /api/webhook-relay endpoint with retry logic.

Phase 3: Database & Drizzle ORM Schema (Day 3)
Explore src/db/schema.ts, src/db/vehicles.ts, and Supabase RLS policies. Learn JSONB querying.

Phase 4: Client Showroom UI & State Management (Day 4)
Examine src/App.tsx router, useMemo filter pipelines, and components InventoryPage.tsx & VehicleGrid.tsx.

Phase 5: Admin Control Panel & Hardware Telemetry (Day 5)
Analyze AdminDashboard.tsx, TelemetryInspector.tsx, AddVehicleModal.tsx, and imageCompressor.ts.

Phase 6: Dual Storage Sync & Production Deployment (Day 6)
Study supabaseVehicleService.ts, real-time WebSockets, Vite build configurations, and caching headers.

Key Takeaways for Your Portfolio

(*) 1. Demonstrates full-stack fluency across React 19, TypeScript, Express, and PostgreSQL.
(*) 2. Solves real-world dealer problems: high-res photo bloat, lead routing, and hardware inspections.
(*) 3. Production-hardened with Gzip compression, retry-enabled webhooks, and hybrid database sync.